9f838a13-4e18-4a0f-8f22-2cb55e8d03

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Homebrew/brew' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/brew/brew (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add </pr_title>`compatibility_version` DSL to Formula class
<pr_description>
Implements a new </pr_description>`compatibility_version` DSL for Homebrew formulae to help minimize unnecessary dependency upgrades when changes are backwards-compatible.

Overview

This PR adds support for formula maintainers to declare API/ABI compatibility levels using a new `compatibility_version` field. When a formula has a specific `compatibility_version`, dependents don't need to be upgraded as long as the installed version has the same compatibility level.

Changes

New DSL Method

Formulae can now declare a `compatibility_version`:

```ruby
class ExampleLib < Formula
  desc "Example library"
  homepage "https://example.com"
  url "https://example.com/lib-1.0.tar.gz"

  # Indicates API/ABI compatibility level
  compatibility_version 1

  def install
    system "./configure", "--prefix=#{prefix}"
    system "make", "install"
  end
end
```

When upgrading from version 1.0 → 1.1 → 1.2 with the same `compatibility_version 1`, dependents won't need to be rebuilt. When the API/ABI changes incompatibly (e.g., version 2.0), bump to `compatibility_version 2`.

Storage and Tracking

The `compatibility_version` is stored in:
- The formula's `INSTALL_RECEIPT.json` in `source.versions.compatibility_version`
- Runtime dependencies for each dependent formula
- Formula serialization via `to_hash`

Dependency Resolution

Updated `Dependency#installed?` to check `compatibility_version` when determining if an upgrade is needed:
- If both installed and required formulae have matching `compatibility_version`, the dependency is satisfied
- Falls back to existing version/revision checking if compatibility versions don't match or are unset

Implementation Details

- Optional field (returns `nil` if not set) for back is bumped whenever needed

2. `compatibility_version` is _not_ bumped when a bump is not needed

To do the above, we would likely need to run the following dependent tests (or something similar to them):

If `compatibility_version` is bumped, we need to check that each dependent is broken when using the previous `compatibility_version`, but is not broken under the newer `compatibility_version`.

If `compatibility_version` is _not_ bumped, then we need to check that each dependent still works when using the oldest bottles that have the same `compatibility_version`.

What is the motivation for the feature?

This should help minimise the number of formulae that `brew {install,upgrade} foo` needs to upgrade to ensure that user installs are not broken.

How will the feature be relevant to at least 90% of Homebrew users?

This should help address a pain point that many users have expressed many times over the years.

What alternatives to the feature have been considered?

Doing nothing.

Comments on the Issue (you are @copilot in this section)

<comments>
<comment_new><author>@MikeMcQuaid</author>
Thanks for write-up @carlocab, this is great! Thoughts:

> Formulae should have an (optional?) </comment_new></comments>`compatibility_version` field.

Suggest "optional while we're rolling it out" and then "mandatory for anything that has a dependency on anything else" (i.e. if something has no dependancies: not required, if a dependency is added later: CI fails on the dependent add until it's added to the dependency).

For an initial value for this field: we could set it to the

`.dylib`/`.so` "major" version for libraries and/or the major version for things that seem at least vaguely semver.

> If `compatibility_version` is _not_ bumped, then we need to check that each dependent still works when using the oldest bottles that have the same `compatibility_version`.

I think this logic is essentially the same way we detect "`revision` bumps" today: we download the bottle, we attempt `brew linkage` and `brew test` on that bottle and, if they both pass, no bump is needed.

The tricky bit is deciding if some dependencies need a bump and others don't (and also the cases where the linkage or test has bitrot into failing). Perhaps it should be something like >=50% of total dependencies needed bumped?

> If `compatibility_version` is bumped, we need to check that each dependent is broken when using the previous `compatibility_version`, but is not broken under the newer `compatibility_version`.

This will be trickier. Might be a bit of a manual/human process here or `brew test-bot` outputs something at an earlier stage that ensures that e.g. a label is set in the PR and a bump is allowed?

Will want to make sure we socialise (very) hard that `compatibility_version` should NEVER be bumped except when `brew test-bot` says that it's needed.

> The `compatibility_version` field should capture suitability for use as a dependency. In particular, if a formula has a `compatibility_version` of `1`, then it need not be upgraded when installing or upgrading dependencies as long as dependencies are known to work with versions of the formula that have `compatibility_version 1`.

Note: this should be stored in the `INSTALL_RECEIPT.json`.<comment_new><author>@carlocab</author> > I think this logic is essentially the same way we detect "</comment_new>`revision` bumps" today: we download the bottle, we attempt `brew linkage` and `brew test` on that bottle and, if they both pass, no bump is needed.

Somewhat. The tricky part is the following. Suppose formula `foo` has versions `1.0`, `1.1`, and `1.2`, and all those

```
  versions have `compatibility_version 1`. In our current CI
  setup, when we bump `foo` to version `1.3` but keep
  `compatibility_version 1`, we test the dependents of `foo`
  against version `1.3`. However, I think we need to test
  dependents at least against version `1.0` as well.

  The ...


  Fixes Homebrew/brew#19202


  ## Comments on the PR (you are @copilot in this section)

  <comments>

  <pr_reviews>

  <comment_thread>
  <file>Library/Homebrew/dependency.rb:74</file>
  <original_commit>4e0dd47</original_commit>
```

```
  <comment_new>
  <comment_id>2411079801</comment_id>
  <author>@MikeMcQuaid</author>
  @copilot Add a </comment_new></comment_thread></
  pr_reviews></comments>`formula.compatibility_version` helper
  method to access this without needing `.class` and use this
  everywhere consistently.


  ----
  The last **2** git commits in this branch are the changes
  you have made so far. Use those as your change commit
  history.
  </problem_statement>
```

```
Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
```

- Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
- Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.

<prev_assistant>

<empty>

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Homebrew/brew' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/brew/brew (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add `compatibility_version` DSL to Formula class
</pr_title>

<pr_description>
- [x] Add `compatibility_version` DSL method to Formula class (similar to `revision` and `version_scheme`)
- [x] Initialize `@compatibility_version` instance variable in Formula#initialize
- [x] Update `formula_to_dep_hash` in Tab class to include `compatibility_version` in runtime dependencies
- [x] Update Tab#create to store `compatibility_version` in source versions
- [x] Update `Dependency#installed?` to check `compatibility_version` for dependency satisfaction
- [x] Add tests for `compatibility_version` DSL in formula specs
- [x] Add tests for `compatibility_version` in Tab specs
- [x] Update Formula#to_hash to include `compatibility_version` in serialization
- [x] Run typecheck, style and tests to ensure all changes work correctly
- [x] Add `formula.compatibility_version` helper method for cleaner access (instead of `formula.class.compatibility_version`)

Fixes Homebrew/brew#19202
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_reviews>

<comment_thread>
<file>Library/Homebrew/dependency.rb:74</file>
<original_commit>4e0dd47</original_commit>

<comment_old>
<author>@MikeMcQuaid</author>
@copilot Add a `formula.compatibility_version` helper method to

access this without needing `.class` and use this everywhere consistently.
</comment_old>

<comment_old>
<author>@copilot</author>
Added `attr_reader :compatibility_version` to Formula class and updated all usages to use `formula.compatibility_version` instead of `formula.class.compatibility_version`. Changes applied in commit a0c9e7e.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>3380203894</comment_id>
<author>@MikeMcQuaid</author>
@copilot fix this CI failure:
```
+ brew install shellcheck shfmt
==&gt; Fetching downloads for: shellcheck and shfmt
==&gt; Downloading https://ghcr.io/v2/homebrew/core/shellcheck/manifests/0.11.0
==&gt; Downloading https://ghcr.io/v2/homebrew/core/shfmt/manifests/3.12.0
Error: Your definition of `installed?` is missing these keyword arg(s): [:minimum_compatibility_version] which are defined in the method it overrides:
 * Base definition: in Dependency at /home/linuxbrew/.linuxbrew/Homebrew/Library/Homebrew/dependency.rb:49
 * Override: in UsesFromMacOSDependency at /home/linuxbrew/.linuxbrew/Homebrew/Library/Homebrew/dependency.rb:280
Error: Process completed with exit code 1.
```
</comment_new>

</pr_comments>

</comments>

The last **3** git commits in this branch are the changes you

```
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to
validate each code change thoroughly.
```